

PRA-SGLang: Query-Addressed Non-Prefix Memory beside RadixAttention

Jorge Simão
EInnovator R&D

August 2026

Abstract

SGLang’s RadixAttention reuses exact token prefixes. Progressive Retrieval Attention (PRA) selects versioned non-prefix resources and intervals according to the current query. We implement a logical PRA object namespace and an SGLang adapter and an MLX runner-cache bridge that keeps PRA memory outside Radix prefix accounting while consuming it as selected native K/V. A 25-request Apple-M5 smoke study with patched SGLang MLX separates exact radix reuse from selected-memory recovery. Prefix plus selected memory and full context both recover the fixed answer in 5/5 requests, while the no-evidence controls are 0/5. Selection uses 365 prompt tokens instead of 1,577. The scheduler reports 364 cached tokens on warm selected requests and 1,576 on warm full-context requests; warm TTFT is 34.1 and 52.7 ms, respectively. In a separate five-seed native-cache experiment, selected K/V recovers 5/5 answers and is bit-exact to SGLang’s ordinary split cache. All 28 attention layers consume selected memory under one normalization, while the scheduler-visible local offset excludes every PRA token. An external PRA HiCache then places immutable detail in attention-ready L1 arrays, host-owned L2 arrays, and compressed NPZ L3 backing without entering the Radix namespace. Across five seeds, answer recovery and Radix separation are 5/5; warm L1, L2-to-L1, and L3-to-L1 resolution average 0.009, 4.0, and 405.6 ms. Forced capacity pressure performs both L1-to-L2 and L2-to-L3 demotion in every seed. The same logical-object cache is then connected to SGLang’s built-in `HiCacheStorage` file backend. Five fresh-adapter reads recover 5/5 answers; raw 14.35 MB blobs write in 38.7 ms on average, restore to L1 in 122.2 ms on average (36.2 ms median), and hit warm L1 in 0.013 ms. An oracle prefetch sweep then rejects Python-thread overlap as a production mechanism: promotion takes 25–28 ms, but requested 10–50 ms leads delay the caller by 193–235 ms rather than overlapping useful work. In a combined runner-level cohort, an 81-token shared-prefix hit and external HiCache selection coexist in one request. Selected A and reselected C recover 5/5, cleanup request B recovers 0/5, and all selected requests contain exactly one memory copy. The bridge also runs through SGLang’s real prefill and batched-decode lifecycle: native and full context recover 5/5, disabled memory recovers 0/5, and native throughput rises from 6.59 to 14.54 requests/s between concurrency one and eight. Shared immutable K/V avoids 108.0 MiB at concurrency eight. Across 40 QASPER and 40 HotpotQA natural-text transport controls per condition, native and full context are 80/80, disabled is 40/80, and shuffled memory is 32/80. On a separate frozen 60-example routed-QA cohort, E0 selected text and E2 native K/V produce exactly the same output in all 840 pairs spanning cold, warm, multi-query, and concurrency-eight schedules while E2 removes 90.6–93.0% of visible prompt tokens. Cold end-to-end cost is slightly lower for E2, but warm, multi-query, and concurrent cost ratios are 1.136, 1.140, and 1.122. Semantic transport is closed; the unfused selected-cache path remains an economic optimization target. A final three-example lifecycle probe routes logical keys through the shared manager and built-in HiCache storage. WARM is 3/3 exact and restart recovery succeeds; int8 COLD changes 3/3 sequences and remains opt-in.

1 Introduction

A prefix cache and a retrieval memory answer different questions. A radix tree asks whether token histories share an exact path. PRA asks which retained resource or interval is useful for a query. A long-running session can use both:

$$\underbrace{\text{stable active dialogue}}_{\text{radix prefix}} + \underbrace{\text{query-selected archive}}_{\text{PRA resource}}.$$

SGLang combines language-model serving with a radix cache and optional hierarchical KV mechanisms [1]. The systems question is not whether selected text can be sent to SGLang; a gateway already does that. It is whether

a stable non-prefix PRA identity can share SGLang’s physical cache hierarchy and scheduling substrate without being misrepresented as a prefix.

This iteration contributes an engine-independent logical object lifecycle, a capability-honest SGLang adapter, a measured RadixAttention baseline, and a native runner-cache bridge measured through live prefill and batched decode. External placement, the built-in SGLang file-storage bridge, and the combined shared-prefix/native path are now measured locally; off-node distributed placement remains open.

2 Two Address Spaces

The two logical keys are

$$a_{\text{radix}} = (t_1, \dots, t_n), \quad a_{\text{PRA}} = (\tau, s, r, v, [i, j], \ell, m, d, p),$$

where the PRA key includes tenant τ , optional session s , resource and version (r, v) , source interval $[i, j]$, layer ℓ , model revision m , layout/dtype d , and positional/materialization policy p . Equality in one space does not imply equality in the other.

The black-box baseline selects exact evidence outside SGLang and serializes it into the current turn. RadixAttention may then cache that resulting token prefix. An engine-aware design would instead retain the PRA logical identity while placing its physical detail in GPU, host, or distributed hierarchy.

3 Implementation Boundary

`src/prahf/engine_memory.py` provides validated states `INDEXEDONLY`, `OFFDEVICE`, `PREFETCHING`, `RESIDENT`, `PINNED`, `EVICTABLE`, and `INVALID`. Resident transitions require physical handles. Selection enforces tenant and authorization scope, and snapshots keep address, logical detail, and resident detail bytes disjoint.

`src/prasglang/adapters.py` exposes the OpenAI-compatible server as E0. It advertises automatic prefix caching and selected-text fallback, but no logical references or native K/V. Supplying a `SGLangNativeExecutor` promotes the adapter to E2 and transfers generation, streaming, session close, resource deltas, and block-store access to the executor. This pattern prevents a sidecar namespace from being mistaken for attention integration.

The hierarchical bridge in `src/prasglang/hicache.py` maps PRA identities to an independent HiCache-style namespace. It does not insert resources into the radix tree as artificial prefixes. Attention-ready MLX arrays form L1, host-owned arrays form L2, and compressed NPZ plus a logical-identity manifest forms L3. LRU pressure demotes L1 to L2 and L2 to L3; lookup promotes in the opposite direction before attention. On unified memory, L1 and L2 describe ownership and readiness rather than physically separate GPU and CPU DRAM.

`src/prasglang/hicache_backend.py` adds an immutable blob protocol over SGLang’s actual `HiCacheStorage` interface. A fixed header carries the payload length required by the backend’s preallocated-read contract; a hashed namespace key hides the resource URI. The same adapter can wrap the built-in file backend or a backend returned by SGLang’s factory. The portable interface does not expose per-key physical deletion, so PRA invalidation revokes logical access and relies on versioned identities plus backend eviction for byte reclamation. Raw framing is the default: an initial compressed transport added 2.8–3.0 seconds of CPU work and was rejected before final measurement.

`src/prasglang/mlx_native.py` implements the first attention bridge. Each layer cache exposes a local `offset` to scheduler and Radix bookkeeping, a distinct `rope_offset` to attention, and concatenated selected-plus-local K/V only inside the attention view. Prefix-pool synchronization sees local K/V arrays, so PRA detail cannot be mistaken for an exact prefix. Hooks cover initial prefill, request cleanup, and the runner’s batched-decode cache protocol. Release-time unwrapping is essential: the first live run showed that returning a selected-memory wrapper to SGLang’s reusable cache pool lets a later request inherit memory and can double-wrap a later native request. The corrected release hook returns only clean request-local K/V to the pool, and explicit geometry checks reject contaminated caches. The external HiCache supplies selected memory to this request-local bridge while remaining disjoint from ordinary reusable cache pools.

4 Environment

The host was an Apple M5 MacBook Pro with 16 GB unified memory, Metal 4, and macOS 26.4.1. SGLang revision `ef20fab38a03490e2cdf1b7377145ca3a3f2bfc5` ran under Python 3.11.15, MLX 0.32.2, and `mlx-lm` 0.31.3. The

Table 1: SGLang MLX smoke results. TTFT is milliseconds. Warm cached tokens come from the scheduler log rather than the SSE usage object.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	511.8	44.3	23
Prefix	0%	333	97.3	31.6	332
Selected	100%	56	40.8	31.5	55
Prefix + selected	100%	365	43.5	34.1	364
Full context	100%	1577	287.4	52.7	1576

Table 2: Five-seed SGLang native-cache mechanism result. Radix separation is the fraction for which scheduler-local length excludes selected PRA tokens.

Seeds	Exact	Argmax parity	Radix separation	Latency ms
5	100%	100%	100%	222.0

checkout required a local compatibility patch that restores the loader property expected by `MlxModelRunnerStub`; the patch is part of provenance, not hidden setup.

The model was `Qwen/Qwen3-0.6B` at revision `c1899de289a04d12100db370d81485cdf75e47ca`. The server used one running request, 2,048 total tokens, disabled CUDA-graph backends, and the MLX runner. Startup identified `UnifiedRadixCache` with LRU eviction. Built-in distributed HiCache was disabled for the HTTP control. A separate experiment uses SGLang’s built-in file backend through `StorageBackendFactory`; it tests the engine storage contract but not off-node transport. The control plane reported `torch.native`, while model inference was delegated to MLX; both facts are retained in the artifact.

The live native-runner experiments use the 4-bit `mlx-community/Qwen3-0.6B-4bit` checkpoint at revision `73e3e38d981303bc594367cd910ea6eb48349da8`. This separates them from the earlier unquantized HTTP smoke and prevents cross-precision latency comparison.

5 Method

We used the same synthetic codeword task and serialized prompts as the vLLM and MLX-LM calibration. Twelve distractor records surround one authoritative record. Conditions are no prefix/no PRA, prefix only, selected evidence only, prefix plus selected evidence, and full context. Each is repeated five times. SSE timing provides TTFT; SGLang scheduler logs provide exact new and cached token counts. Tail percentiles are not reported from five samples.

This HTTP matrix is SMOKE evidence. Separate five-seed controlled experiments exercise native prefill, decode, request cleanup, concurrency 1–8, natural-text causal controls, and external L1/L2/L3 placement. They do not measure off-node cache affinity.

6 Results

The controls behave correctly: no-evidence conditions are 0/5, and every selected or full-context request recovers the answer. Prefix plus selected memory uses 365 tokens, 76.9% fewer than full context. Its cold TTFT is 43.5 ms versus 287.4 ms; its warm mean is 34.1 versus 52.7 ms.

The cache trace is the strongest mechanism observation. After the first request in each condition, the scheduler reports one new token and 364 cached tokens for prefix plus selected memory. Full context similarly uses one new and 1,576 cached tokens. Selected context therefore does not interfere with exact radix reuse. It also creates a much smaller cached sequence. These numbers measure token-prefix cache reuse, not PRA-resource reuse.

7 Native Runner Results

The native path recovers all five answers, preserves next-token logits exactly, and keeps PRA tokens out of scheduler prefix length in every case. Mean completion latency is 222.0 ms. The SGLang MLX control plane reports no

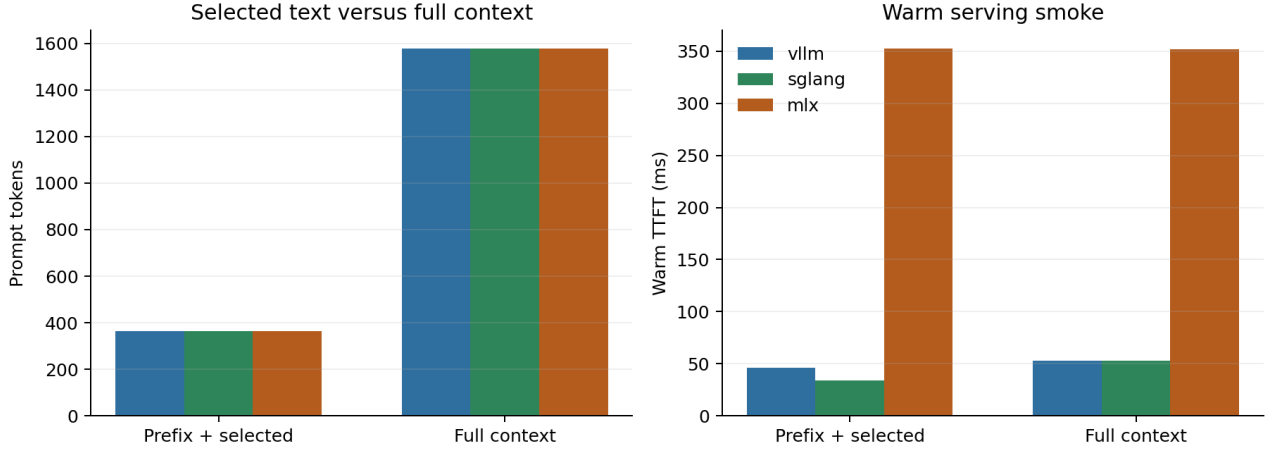


Figure 1: Shared calibration view. Prompt sizes are identical across the Qwen tokenizers. Absolute engine latency is not comparable because SGLang used an unquantized model while vLLM and MLX-LM used a 4-bit checkpoint.

ordinary GPU KV pool because MLX owns inference state.

The next experiment installs the bridge on the real `MlxModelRunner`. It executes source ingestion separately, then runs selected memory through `prefill_start`, `prefill_finalize`, and batched decode. Full visible context and native memory recover 5/5 answers, while query-only disabled memory recovers 0/5. Every native request keeps all 141 source tokens outside the scheduler-local length.

Table 3: Five-seed live-runner concurrency. One immutable selected memory is shared by every request in a batch.

Concurrency	Exact	Requests/s	Shared MB saved	Scheduler isolation
1	100%	6.59	0.0	yes
2	100%	9.91	15.4	yes
4	100%	13.25	46.3	yes
8	100%	14.54	108.0	yes

Table 3 shows exact recovery at every tested concurrency. Mean throughput rises from 6.59 requests/s at one request to 14.54 at eight, with diminishing returns after four. Physical object sharing avoids 108.0 MiB of duplicate selected K/V at concurrency eight. These are controlled in-process measurements, not queued HTTP tail latency.

7.1 Natural-text controls

We also use balanced QASPER and HotpotQA text to build 40 examples per dataset and condition. Each source contains an inserted `AnswerCode yes/no` anchor among natural text. Constrained greedy decoding ranks the two answer tokens through SGLang’s own logit-edit path. This measures native transport and causal dependence, not unrestricted benchmark QA.

Table 4: SGLang live-runner natural-text transport controls.

Dataset	Condition	Samples	Ranked exact
qasper	disabled	40	50%
qasper	native	40	100%
qasper	ordinary_full	40	100%
qasper	shuffled	40	40%
hotpotqa	disabled	40	50%
hotpotqa	native	40	100%
hotpotqa	ordinary_full	40	100%
hotpotqa	shuffled	40	40%

Native and ordinary full context are 40/40 on each dataset. Disabled memory is 20/40 and shuffled memory is 16/40 on each. The negative controls establish that successful native decisions depend on the registered source rather than the query alone.

7.2 Matched selected text versus native K/V

We next freeze the routed selections for 20 original-answer examples from each of QASPER, HotpotQA, and 2WikiMultiHopQA. E0 writes the selected source once to ordinary Radix-owned prefix slots. E2 encodes exactly the same source tokens as immutable native K/V and leaves them outside Radix accounting. Both paths use the same query suffix, greedy decoder, and 24-token output budget. The shared schedule includes cold one-shot, two warm repeats, three query formulations over the same resource, and an eight-request wave with independent prefill and batched decode.

Table 5: Matched-selection quality on the shared three-engine cohort. All parity pools 840 request pairs per engine across four schedules.

Engine	Dataset	Visible E0/E2	Reduction	Cold F1 E0/E2	Cold parity	All parity
mlx-lm	2wikimultihopqa	375/33	91.1%	0.067/0.067	100.0%	100.0%
mlx-lm	hotpotqa	415/39	90.6%	0.087/0.087	100.0%	100.0%
mlx-lm	qasper	399/28	93.0%	0.110/0.110	100.0%	100.0%
sglang-mlx	2wikimultihopqa	375/33	91.1%	0.074/0.074	100.0%	100.0%
sglang-mlx	hotpotqa	415/39	90.6%	0.084/0.084	100.0%	100.0%
sglang-mlx	qasper	399/28	93.0%	0.105/0.105	100.0%	100.0%
vllm-metal	2wikimultihopqa	378/33	91.2%	0.079/0.079	100.0%	100.0%
vllm-metal	hotpotqa	417/39	90.6%	0.074/0.074	100.0%	100.0%
vllm-metal	qasper	400/28	93.0%	0.101/0.101	100.0%	100.0%

Table 6: Macro E2/E0 execution-cost ratios; lower is better. Cold includes ingestion, concurrent is inverse requests/s, and E2 usable is milliseconds.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.130	1.155	1.148	1.132	51.2
sglang-mlx	0.976	1.136	1.140	1.122	55.0
vllm-metal	0.985	1.004	0.993	0.981	86.0

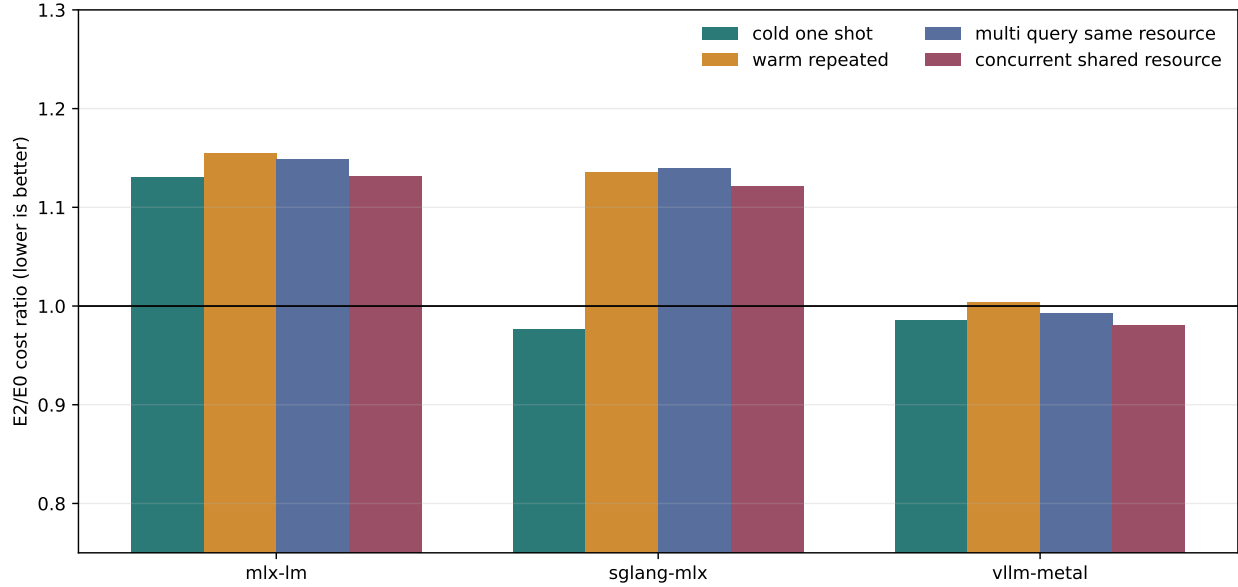


Figure 2: Macro representation cost after freezing retrieval. Lower is better; concurrent cost is inverse throughput. SGLang wins the ingestion-inclusive cold regime but pays unfused reuse and decode overhead.

SGLang preserves exact output and F1 on all 840 E0/E2 pairs. E2 reduces visible prompt tokens by 90.6–93.0%. Its macro cold end-to-end cost ratio is 0.976 after 55.0 ms mean time to usable context: native ingestion offsets the slower online path for a one-shot request. Warm and multi-query ratios rise to 1.136 and 1.140. At concurrency eight, E2 reaches 89.2% of E0 throughput, an inverse cost ratio of 1.122. The wrapper concatenates selected and local arrays at each consumer layer and decode step, so native transport is semantically closed while fused materialization and decode economics remain open.

7.3 Semantic tiers versus HiCache tiers

The shared runtime now owns HOT/WARM/COLD/SOURCE as semantic service levels. For this paper, HOT maps to the attention-ready PRA object, WARM can map to a lossless host or HiCache L2 representation, and COLD can map to the built-in file backend or another validated L3 provider. This does not merge PRA identity with Radix identity. HiCache supplies physical storage and transport; the shared manager supplies typed-record priors, task/dependency retention, persistence grace, and deterministic quota eviction.

A common 20-run policy control (four engine labels, five seeds) preserves exact lossless tier round-trip and SOURCE reconstruction, retains open-task and dependent records in every run, and avoids all 20 immediate writes for a transient closure trace. Those are policy and byte-transport checks, not SGLang generation or distributed-HiCache measurements. The live runner, combined Radix/HiCache lifecycle, and matched E0/E2 rows below remain the engine evidence.

A new live bridge makes that ownership executable end to end. The shared manager uses an attention-ready `SGLangPRAHiCache` object for HOT and SGLang’s built-in `HiCacheStorage` file API for WARM. Request registration supplies only logical keys; the bridge promotes, pins, attaches exactly once, unwraps PRA detail before returning local caches to the ordinary pool, and unpins on cleanup. Three natural QASPER selections reproduce HOT output exactly after WARM promotion, and a fresh manager recovers all persisted objects after restart.

For SGLang, median lifecycle latency is 279 ms HOT and 529 ms WARM (1.90×); background WARM placement costs 158 ms median. Int8 COLD promotion plus generation is 527 ms, while background conversion costs 701 ms. Int8 changes all three sequences and preserves the first token in two, so it remains an explicit diagnostic rather than a default HiCache policy. This closes lifecycle-manager control over the local built-in backend, not distributed off-node HiCache or concurrent tail curves.

Table 7: Shared three-example live storage probe. Ratios include promotion and generation but exclude background tier transitions.

Engine	WARM exact	int8 exact	int8 first	Prefix	WARM/HOT	COLD/HOT
mlx-lm	100%	0%	67%	1.3	1.97	1.84
sglang-mlx	100%	0%	67%	0.7	1.90	1.89
vllm-metal	100%	0%	67%	4.0	1.96	2.00

Table 8: Five-seed external PRA HiCache result. L1/L2 on Apple unified memory denote attention readiness and host-array ownership; L3 is compressed disk backing.

Path or invariant	Mean latency ms	Success
Warm L1 hit	0.009	—
L2 $\rightarrow$ L1	4.0	—
L3 $\rightarrow$ L1	405.6	—
Answer recovery	—	100%
L1/L2 pressure demotion	—	100%
Radix namespace separation	—	100%

7.4 External HiCache placement

Table 8 resolves the same immutable selected K/V from each tier before live generation. All five outputs recover the target and all five keep selected tokens outside Radix prefix identity. Warm attention-ready lookup is 0.009 ms on average. Host-to-attention-ready promotion averages 4.0 ms. Compressed L3 restoration averages 405.6 ms and includes one 984 ms outlier, so it is a backing path rather than a latency-neutral cache hit. With one-object L1 and L2 budgets, inserting two additional resources forces one L1-to-L2 and one L2-to-L3 demotion in every seed. This closes local hierarchy mechanics; it does not establish distributed HiCache or server tail latency.

7.5 Built-in SGLang storage backend

The next control replaces the private NPZ L3 path with SGLang’s registered `file` backend while retaining the independent PRA logical namespace. For each seed, one adapter writes an immutable object and a fresh adapter, without process-local placement metadata, checks the backend and promotes it through L3 to attention-ready L1 before generation.

Table 9: Five-seed SGLang HiCacheStorage file-backend bridge. Times are milliseconds; the cold-read mean includes one 476.6 ms outlier.

Measure	Mean	Median	Result
raw object write	38.7	39.0	5/5 stored
cold backend read $\rightarrow$ L1	122.2	36.2	5/5 hits
warm L1 lookup	0.013	0.012	5/5 hits
generation after restore	—	—	5/5 exact

Each source has 125 tokens and 14.336 MB of native K/V; framing adds 17.2 KB. The result closes compatibility with SGLang’s real storage abstraction and fresh-adapter discovery. It does not establish Mooncake, NIXL, HF3FS, AIBrix, shared-memory, or cross-host transfer. Those backends can now be selected without changing PRA serialization or logical identity, but each requires its own measured correctness, transfer, and cleanup gate.

7.6 Prefetch overlap diagnostic

The storage bridge exposes a caller-owned prefetch hook so an engine scheduler can initiate promotion before a selected object is demanded. We evaluate an oracle signal, which removes selection uncertainty and gives the mechanism its most favorable timing case. Each condition contains five seeds and promotes a 14.336 MB, 125-token native-K/V object from the built-in file backend to attention-ready L1.

Table 10: Caller-owned Python-thread prefetch. “Actual lead” is elapsed time before the caller regains control and issues the demand. All 20 restored tensors are exact. Times are milliseconds.

Requested lead	Promotion	Actual lead	Ready at demand	Demand stall
0	27.9	0.4	0/5	27.6
10	27.6	203.3	5/5	0.094
25	25.0	192.6	5/5	0.088
50	26.3	235.3	5/5	0.109

The near-zero demand stalls at nonzero requested leads are not an optimization result. Python deserialization and MLX array construction delay the caller, so the nominal 10–50 ms scheduling window expands to 193–235 ms. The experiment closes exact asynchronous API semantics but leaves useful overlap open. A production path must use SGLang’s native asynchronous HiCache controller, a GIL-independent worker, or an equivalent engine-owned transfer path before prefetch latency claims are warranted.

7.7 Combined shared-prefix and external-memory lifecycle

Table 11: Five-seed A/B/C lifecycle through SGLang’s shared-prefix pool and external PRA HiCache. Ordinary B should not recover the selected fact.

Request	Recovery	Radix separated	One copy	Mean ms
A: selected after L2 promotion	5/5	5/5	5/5	196.7
B: ordinary after cleanup	0/5	5/5	—	127.8
C: reselected from warm L1	5/5	5/5	5/5	147.2

Table 11 drives the runner interface used after an exact Radix match. A warm request writes 81 stable-prefix tokens into SGLang’s shared K/V pool. Request A gathers those prefix slots while the PRA bridge promotes its separately named memory from L2 to attention-ready L1. Request B uses the same shared prefix after A is removed, but receives no PRA identity. Request C selects that identity again from warm L1. A and C recover 5/5; B recovers 0/5. Scheduler-local length excludes PRA in all 15 requests, and each selected cache reports the source token count once rather than twice.

This closes the local composition and cleanup mechanism. It also exercises capacity pressure across the five resources: three L1-to-L2 and six L2-to-L3 demotions occur while preserving recovery. The experiment directly controls the prefix-slot handoff rather than entering through SGLang’s HTTP scheduler; distributed HiCache, scheduler affinity, and online tail latency remain open.

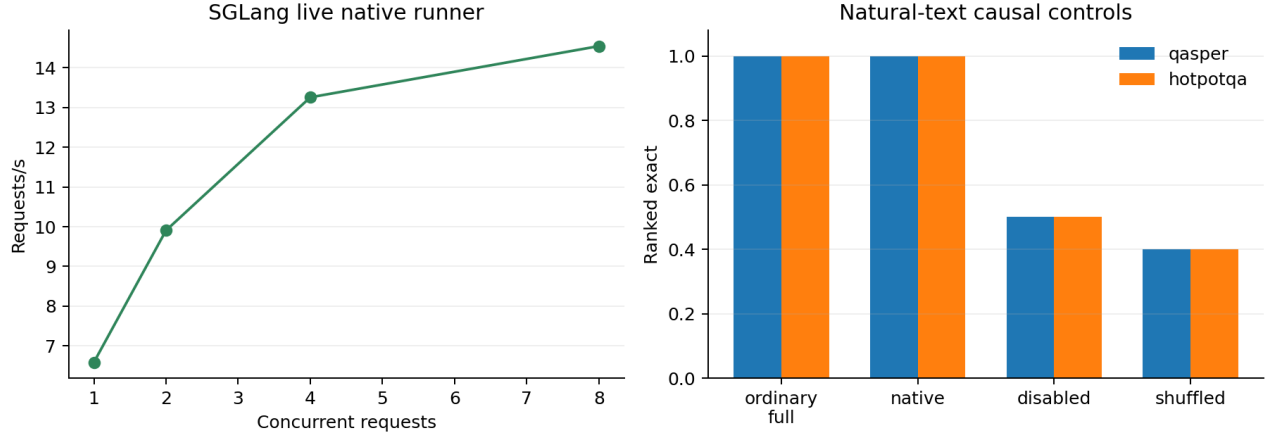


Figure 3: Live native-runner throughput and natural-text causal controls.

Gate	Status	Evidence
environment	closed	patched pinned SGLang-MLX starts reproducibly
radix baseline	smoke closed	exact scheduler cache-token trace
selected-text parity	smoke closed	selected and full are 5/5
HiCache non-prefix object	controlled closed	five-seed L1/L2/L3 promotion and forced demotion
built-in HiCache storage	controlled closed	file backend, fresh adapter 5/5, restored generation 5/5
shared storage lifecycle	controlled closed	logical-key promotion; WARM 3/3 exact; restart recovery; int8 COLD open
prefetch overlap	negative/open	exact 20/20; 10–50 ms requests became 193–235 ms caller delays
selected native attention	controlled closed	5/5 exact, zero logit error, all 28 layers
radix/native identity	controlled closed	PRA tokens excluded from local prefix length
Radix + HiCache request	controlled closed	A/C 10/10, cleanup B 0/5, exactly one selected copy
runner lifecycle	controlled closed	prefill, batched decode, and clean request release
concurrency/sharing	controlled closed	five seeds at 1–8 requests; 108 MiB avoided
natural transport	controlled closed	native/full 80/80; causal controls near chance
matched natural QA	controlled closed	840/840 exact E0/E2 outputs; 90.6–93.0% fewer visible tokens
robustness	partial	one model; synthetic and two natural-text transport fixtures

8 Next Engine-Native Experiment

The matched single-request comparison is now complete. The next useful patch moves the verified local composition into off-node HiCache placement and scheduler affinity. The Python-thread control shows that useful prefetch overlap requires SGLang-native asynchronous transfer rather than a host-thread wrapper. Concurrent cold/warm tier curves and server p50/p95/p99 should follow that integration. Only then should learned placement or cache-aware scheduling be optimized.

9 Falsification and Limitations

Deep SGLang integration is unnecessary if RadixAttention plus the tuned black-box path matches native PRA on quality-adjusted memory, TTFT, and throughput. The current baseline is already strong, so a metadata-only object registry cannot satisfy the claim.

The measurements use one small model family, a local compatibility patch, and one Apple host. The HTTP and native runs use different model precisions. The broad concurrency and natural-text runner cohorts disable

Radix pooling; the dedicated combined cohort instead drives the exact prefix-slot boundary directly, without the HTTP scheduler. Distributed HiCache, HTTP-level native concurrency, tail latency, resource-affinity routing, and archived-head transition remain unmeasured. L3 timing includes only five loads and one large outlier. The natural controls contain real dataset text but an inserted answer-code task. The matched cohort uses original benchmark answers, but only 20 routed selections per dataset and a 24-token greedy decode. Its exact transport parity is stronger than its low absolute F1. No distributed HiCache or queued server result is implied.

10 Related Work

SGLang combines a structured runtime with RadixAttention for shared prefix reuse [1]. Paged serving systems manage physical KV, while hierarchical caches extend residency across tiers. Retrieval systems select text or documents before generation. PRA differs by assigning stable identity to native non-prefix memory and making its active subset query dependent. The central integration constraint is to preserve both prefix and resource identities rather than collapse one into the other.

11 Conclusion

On the pinned SGLang-MLX environment, selected context and RadixAttention are clearly complementary: the selected condition preserves 5/5 recovery, removes 76.9% of visible prompt tokens, and reuses 364 of 365 tokens on warm requests. The native SGLang-cache path additionally gives 5/5 recovery and exact logit parity without adding PRA tokens to Radix prefix length. The live runner keeps that result through batched decode, scales to 14.54 requests/s at concurrency eight, and passes 80/80 natural-text transport decisions while disabled and shuffled controls fall near chance. SGLang attention can therefore consume selected non-prefix K/V while preserving the two address spaces. Paper 6.1 also establishes local L1/L2/L3 promotion and forced demotion without entering the Radix namespace and uses SGLang’s built-in file storage through a fresh adapter with 5/5 recovery. A final A/B/C cohort composes the shared-prefix pool with external HiCache, preserves 10/10 selected recoveries, prevents 5/5 cleanup leaks, and attaches exactly one memory copy. A five-seed prefetch diagnostic recovers all 20 tensors exactly but finds no useful Python-thread overlap: the caller is delayed by 193–235 ms. It leaves distributed HiCache, native asynchronous prefetch, full-scheduler integration, and pressure-scale server validation as the focused next experiments. On matched routed QA, it preserves all 840 E0 outputs while moving 90.6–93.0% of prompt tokens to native memory; the measured decode overhead makes fused selected-cache consumption the immediate systems optimization.

References

- [1] L. Zheng et al. SGLang: Efficient Execution of Structured Language Model Programs. 2024.